Bit and Word Transposition on Tapes

Hans Brende GPT-6 Astra (OpenAI)

October 2026

Abstract

We prove that linear-time transposition of dyadic bit squares implies linear-time transposition of canonical square matrices with logarithmic-width entries on deterministic multitape Turing machines. One fixed wrapper recovers all dimensions and parameters from raw input, and the assumed bit machine may have an adaptive schedule and nonlinear workspace encodings. We also prove barriers for restricted compilers in the reverse direction. A multiscale bound limits the transpose coordinates covered by a bounded family of bounded-length paths through public interval permutations, whole-entry operations, rotations, reversals and parity rearrangements. A second theorem allows finitely many synchronous nonlinear scans over a fixed number of banks, with bounded persistent state. One common helper records exceptional values and boundary states; acyclic decoding yields a finite inequality excluding all sufficiently large dyadic transposes in this grammar. The unrestricted converse and unrestricted linear-time transposition remain open. All stated theorems have Lean proofs in a shared framework with our earlier papers.

Contributions. Hans Brende directed the research, supplied sources, and reviewed the scope and interpretation of results. GPT-6 Astra (OpenAI) developed the arguments and Lean proofs and prepared the manuscript under his direction.

Contents

1 Introduction **2**

2 Model and two transposition problems **2**

3 The bit-to-word reduction **3**

3.1 A constant-length address factorization 3

3.2 Native execution and workspace reuse 4

3.3 Uniformity and arbitrary dimensions 4

4 A multiscale obstruction for interval compilers **5**

5 Adding synchronous nonlinear scans **7**

6 Consequences and remaining questions **9**

A Formalization and reproduction **10**

1 Introduction

Matrix transposition exposes a distinction between identifying the required permutation and paying to execute it. The distinction is especially sharp on a machine with a fixed number of tapes: a short description of a permutation does not by itself supply an efficient tape computation.

Our earlier papers established a native transposition model and studied local coding and tape geometry within that model [4, 5]. The present work separates two questions. First, can a linear-time bit transposer supply the word transposer used in the canonical problem? Second, which restricted compilers could run in the opposite direction? The constructive analysis uses a constant-length factorization of binary address permutations. The negative analysis charges both exceptional word operations and information carried across scan boundaries.

The barriers are statements about explicitly defined compiler grammars. They remain valid when public operation descriptions vary with the dimension and when permitted scan transitions are nonlinear. They are not normal-form theorems for arbitrary multitape computations. In particular, they do not resolve whether a more general word-to-bit reduction exists.

Harvey and van der Hoeven study reductions from transposition to integer multiplication in the multitape model [7]. Section 7.6 of their 2025 version identifies the difficulty of transferring complexity bounds between bit and multi-bit entries, and gives a bit-to-word simulation for oblivious machines. Our reduction concerns the canonical logarithmic width and the linear-time regime, while allowing an arbitrary adaptive bit machine. It does not assert their more general comparison across all widths and time bounds. The converse compiler barriers here also do not strengthen their multiplication-based transposition algorithms: an unrestricted transposition lower bound would still be required. These distinctions are necessary when interpreting progress on the original multiplication problem.

The underlying methods have established precedents. Address-bit factorizations occur in out-of-core transposition, for example in [9, Section 4]; history-and-replay cleanup is due to Bennett [3]; and our shared-helper bound is an application of the acyclic index-coding bound [2]. The constructive result combines its specific factorization with paid native execution to remove an obliviousness assumption in the canonical linear-time setting.

The negative results concern different restrictions from earlier lower bounds. Kirchherr's conservative-tape result treats entries as indivisible pebbles [8]. Adler et al. connect oblivious I/O transposition with a network-coding conjecture [1, Section 6]. Our scan bound allows nonlinear mixing and requires no coding conjecture, but restricts geometry, synchronization and state. It does not strengthen either result in its original model. Its contribution is a common-scale bound and a fully charged helper derived from the concrete scan semantics.

2 Model and two transposition problems

We use the deterministic multitape model of [4], imported through the shared Lean package of [5]. A machine has fixed finite control, a fixed finite work alphabet, and a fixed number of two-way work tapes. Initially every work cell is blank and every head is at the origin. Its read-only input is a raw finite bit string; the input end symbol is visible outside the string. Output is append-only. One transition queries the scanned symbols, updates the finite control and work symbols, moves each head by at most one position, and appends at most one output bit. No matrix header or dimension is supplied separately.

For $d \geq 1$, the canonical entry width and input length are

$$b(d) = \max\{1, \lceil \log_2 d \rceil\}, \qquad P(d) = d^2 b(d).$$

The Lean definition is `Transpose.entryBits`, namely `Nat.log2 (d - 1) + 1`; Lean's natural-number logarithm satisfies `Nat.log2 0 = 0`. A matrix is serialized in row-major order, with the bits of each entry consecutive. The desired output uses the same bit order within each transposed entry. These are `Transpose.encode` and `Transpose.transpose`.

The statement WORD is `Transpose.LinearTimeTransposable`: there are one native machine M and one constant $c \in \mathbb{N}$ such that every canonical input of every dimension $d \geq 1$ is transposed within $c(P(d) + 1)$ transitions. The statement BIT asks for one native machine and one constant that transpose every square bit matrix of dyadic side 2^h in at most $c4^h$ transitions, for every $h \geq 0$, as in `BitWord.DyadicBitOracle`. These quantifiers require one machine for all sizes.

The compiler models in Sections 4 and 5 are separate finite models used to study possible reductions in the opposite direction. Their operations are defined directly; there is no assertion that every native computation has one of those forms. In particular, charging a word-oracle call does not turn arbitrary surrounding computation into a permitted stage.

3 The bit-to-word reduction

Theorem 3.1 (Uniform bit-to-word reduction)**.** *If one deterministic multitape machine transposes every dyadic square bit matrix in time linear in its input length, then one such machine transposes every canonical square matrix with entry width $b(d)$ in linear time:*

$$\mathsf{BIT} \Longrightarrow \mathsf{WORD}.$$

Both problems use the raw-input model of Section 2. The assumed bit machine need not have an oblivious schedule or a workspace-reset guarantee.

The Lean endpoint is `BitWord.linearTimeTransposable_of_dyadicBitOracle`. The remainder of this section outlines its proof, from address identities to native execution.

3.1 A constant-length address factorization

For a square of dyadic side $D = 2^h$ with $h \geq 1$, set

$$u = \lceil \log_2 h \rceil, \quad e = u \bmod 2, \quad w = u + e, \quad N = 2h + w = 2n, \quad W = 2^w.$$

The even choice of w makes $n = h + w/2$ an integer and the padded bank length a square power of two. Pad every h-bit entry with trailing zeros to width W. The padded bank has 2^N bits, and each address has fields

$$(\mathrm{row}[h], \mathrm{column}[h], \mathrm{lane}[w]).$$

The desired permutation exchanges the row and column fields and preserves every lane. The bound $W < 4h$ limits the padding cost.

Let S_k exchange the last two k-bit address fields, fixing all earlier positions; it is legal when $2k \leq N$. Let R move the last address bit to the front, and let R^{-1} reverse this rotation. These are source-motion permutations: a bit initially at address a moves to the transformed address. Composition lists below are chronological.

Lemma 3.2 (At most thirteen stages)**.** *For every $h \geq 1$, a specified list of at most thirteen legal operators S_k, R, R^{-1} exchanges the two leading h-bit fields and fixes all tail positions. All indices in the list are explicit functions of h.*

Proof outline. For $h < 32$, one has $w \le 6$. Use w copies of R, then S_h, then w copies of R^{-1}. This uses $2w + 1 \le 13$ stages.

For $h \ge 32$, the chosen parameters satisfy $3w \le n$. Define the five-stage word

$$Q = (S_n, S_{n-w}, S_w, S_{n-w}, S_n).$$

It exchanges the first and last w address positions and fixes the middle. Conjugating the suffix exchange by this endpoint exchange gives

$$(Q, S_h, Q),$$

which has eleven stages. To check the conjugation, split the leading fields as $A = A_0A_1$ and $B = B_0B_1$, where $|A_0| = |B_0| = w$, and write the tail as C. The three operators act by

$$\begin{aligned}(A_0, A_1, B_0, B_1, C) &\xrightarrow{Q} (C, A_1, B_0, B_1, A_0)\\ &\xrightarrow{S_h} (C, B_1, A_0, A_1, B_0)\\ &\xrightarrow{Q} (B_0, B_1, A_0, A_1, C).\end{aligned}$$

This is the desired head-field swap. The endpoint exchange is its own inverse. The finite-address identities, including every boundary case, are checked in `five_suffix_end`, `elevenSuffix_eq_head`, and `canonicalGates_correct`. Their bank interpretation is `canonical_bank_action`. □

3.2 Native execution and workspace reuse

A suffix stage S_k partitions the bank into consecutive blocks of 4^k bits. Each block is a raw $2^k \times 2^k$ bit matrix, so one call to the bit transposer realizes the required suffix exchange. Calls are sequential, reusing one fixed collection of work tapes. The number of calls is not bounded by thirteen: thirteen bounds the number of whole-bank stages. The block lengths sum to the bank length, so linear cost per block gives linear cost per stage.

The oracle may branch on data, move its heads adaptively, and use nonlinear work encodings. The wrapper interprets its actual transition function on a copied raw block and captures the append-only output on a work tape. The cleanup follows the history-and-replay method of Bennett [3], with a checked finite controller for the present model. To restore the workspace, a fixed finite record alphabet stores the old scanned symbols and control label of each simulated microstep. Replaying these records backward restores all unretained tapes and head positions while retaining the captured output. Record creation, replay, erasure, and the return to the origin are included in the bound. The alphabet and number of log tracks may depend on the fixed oracle machine; they do not depend on the block size.

The rotation R is a stable even/odd split: read the bank once, separate the two parities, rewind, and concatenate. The inverse interleaves its two halves. The checked split and interleave subroutines initialize and erase their scratch tapes and restore all heads. A physical ruler controls width expansion, cropping, and block traversal. Moving and resetting a ruler is charged to the data it accompanies. A dimension or a loop index written in binary is not treated as a constant-time arithmetic register.

3.3 Uniformity and arbitrary dimensions

The wrapper has one fixed controller. Its tape count, alphabet and control states may depend on the chosen bit machine, but not on the input dimension. The input contains only the matrix bits. The raw dimension loader from [5] recovers d and the entry width $h = b(d)$.

Set $D = 2^h$ and use the lane width $W = 2^w$ defined above. This choice satisfies

$$d \le D \le 2d, \qquad h \le W < 4h, \qquad V := D^2 W \le 16P, \qquad P = d^2 b(d).$$

The same procedure handles $d = 1$: it pads to $D = 2$, executes the permutation, and crops. No exceptional machine or separate size advice is needed.

The fixed controller performs the following operations in order.

1. Load the raw input onto a marked work tape. Compute binary words for d, h, w, n and the gate parameters, and write thirteen physical opcode and parameter slots. Unused slots contain an idle opcode.
2. Construct unary rulers for row counts and record widths. Expand each entry from h to W bits, then pad rows and columns from d to D.
3. Construct a half-bank ruler and execute the thirteen slots. Every suffix slot computes its block ruler from its stored parameter and reuses the same oracle workspace for all consecutive blocks.
4. Crop rows and columns, then crop entries to h bits. Append the result to the raw output and halt.

The descriptor generator uses a fixed finite table for $h < 32$ and binary arithmetic for $h \ge 32$. Both branches belong to the same native program. Sharing a physical port between two subroutines is a fixed wiring choice in the controller; it neither copies data nor changes its location during execution.

The setup trace `NativePrepare.complete_trace` costs at most $40000P$. Each padding or cropping pass costs at most $200V + 200$ by `NativeNormalize.complete_trace`. The half-bank ruler has cost linear in V and its binary exponent length. Each whole-bank gate costs at most

$$(\kappa + 300)V + 24\ell + 100,$$

where κ depends only on the fixed bit machine and its linear-time constant, and ℓ is the stored parameter-word length. `NativeStages.complete_trace` executes all thirteen slots and includes their linking transitions. The generated parameter words have length at most $2h + 2$, so these costs sum to $O(P)$. This accounting includes physical counter construction, repeated block calls, cleanup and raw output; it does not charge a binary arithmetic operation as a single tape transition.

The semantic composition uses exact serialization identities. Padding commutes with transpose, as does widening every entry by trailing zeros. The checked address action therefore transforms the padded serialization into the padded serialization of the transposed original matrix. Cropping recovers precisely `Transpose.encode d (Transpose.entryBits d) (Transpose.transpose A)`. The complete trace is `NativeReductionMachine.complete_trace`. It proves the bound $(200000 + 208\kappa)P$ from blank work tapes, with the exact raw transposed output and actual halting. The compiler bridge identifies its initial configuration with the original machine model's initial state and yields `NativeReductionMachine.native_transpose`, from which Theorem 3.1 follows.

4 A multiscale obstruction for interval compilers

We next examine a restricted converse to the bit-to-word reduction. The operations below include the endpoints of actual canonical word-transpose calls. Their internal computations may be adaptive: only their resulting whole-entry permutations enter the argument. The restriction is on the surrounding compiler and its public layout, not on how an oracle implements its endpoint. This complements the local coding and tape-geometry barriers in [4, 5].

Definition 4.1 (Permitted stage)**.** A bank has addresses $0, \dots, P-1$. A permitted stage is either a stable split into even and odd addresses, its inverse, or the following operation on any public family of disjoint contiguous intervals. Each interval independently carries one of these operations:

(i) an arbitrary permutation of n consecutive k-bit entries, preserving the offset within each entry, where $k, n \geq 1$ and $kn \leq V(k) := k4^k$;
(ii) a cyclic rotation by any fixed offset;
(iii) reversal.

The intervals may have different widths, lengths and starting addresses. Uncovered addresses are fixed. The parity split requires even P. All choices are independent of the bank contents.

The formalization constructs the endpoint permutation and its inverse from these interval and operation data; it does not assume the geometric conclusion below. A canonical word block of side r and width $b(r)$ is included because $r \leq 2^{b(r)}$, hence $b(r)r^2 \leq V(b(r))$. `PartitionData.toStage` and `canonicalWordBlock_admissible` establish these two facts.

Fix a positive integer B and partition the bank into aligned bins of B addresses. An entry-permutation interval of width k is *bad at scale* B when

$$k < B < V(k).$$

Its whole interval is then marked bad. Rotations, reversals and parity operations have no bad addresses.

Lemma 4.2 (One-stage geometry and exceptional volume)**.** *For any permitted stage, the good addresses in one B-bin have images in at most 11 B-bins. The same assertion holds for the inverse stage. If positive scales $B_0, \dots, B_{m-1}$ satisfy $V(B_i) \leq B_j$ whenever $i < j$, the sum of the stage's bad volumes over all these scales is at most P.*

Proof outline. A good entry block either has total length at most B, or has entry width at least B. The small blocks meeting one source bin keep their outputs within three neighboring bins. At most two disjoint blocks of length greater than B meet that source bin. In a large entry block, the source bin meets at most two entries, each translated to at most two output bins. A rotation splits into at most two translated pieces; a reversal has at most two output bins. Including the uncovered addresses, these estimates give $3 + 2 \cdot 4 = 11$. Stable split and interleave meet at most four and six bins, respectively, and therefore satisfy the same bound. All estimates apply in both directions.

A fixed width k can be bad at at most one separated scale. Summing the volumes of the disjoint intervals therefore charges each address at most once. The Lean results are `Stage.good_image_bins_le`, `Stage.predecessors_card_le`, and `Stage.total_bad_volume_le`. □

A *path* is a chronological composition of permitted stages. It may depend on the matrix dimension. Let $\gamma_1, \dots, \gamma_K$ be paths of length at most L on $P = d^2$ bits, and let T be the row-major bit transpose permutation. Define the set of covered source addresses by

$$C = \{u : \gamma_j(u) = T(u) \text{ for some } j\}.$$

Coverage is an endpoint coincidence, not a probability that the compiler is correct on a random input.

Theorem 4.3 (Common-scale path coverage)**.** *Let $m \geq 1$ and let B_i be positive divisors of d, with $B_i \geq B_* > 0$ and $V(B_i) \leq B_j$ for $i < j$. Then*

$$\frac{|C|}{P} \leq \frac{KL}{m} + \frac{K11^L}{B_*} \qquad (d \geq 1). \tag{1}$$

Proof outline. At a fixed scale, mark an original address dirty if its path visits a bad interval. Since each preceding path prefix is a permutation, pulling bad sets back preserves their cardinalities. Summing over all scales and all K paths costs at most KLP. Thus one scale has total dirty volume at most KLP/m.

The clean part of one source bin reaches at most 11^L bins along each path. Because $B \mid d$, the transpose sends the B addresses of one source bin into distinct output bins. A path therefore agrees with the transpose on at most 11^L clean addresses per source bin. There are P/B source bins. Adding dirty addresses and taking the union over paths gives (1). The compiled endpoint is `Compiler.transpose_coverage_bound`; it states the equivalent unnormalized inequality and also covers the empty bank. □

Corollary 4.4 (Pointwise nonlinear readout)**.** *For every fixed K, L, there is a positive dyadic integer d_0 such that, for every positive dimension d divisible by d_0, no K paths of length at most L, followed by arbitrary public address-dependent Boolean tables, transpose every bit input. In particular, this excludes every dyadic $d \geq d_0$. Specifically, an output of the form*

$$y_a = f_a(x_{\gamma_1^{-1}(a)}, \ldots, x_{\gamma_K^{-1}(a)})$$

cannot equal $x_{T^{-1}(a)}$ for every x and a.

Proof outline. Choose $m \geq \max\{1, 4KL\}$ and a dyadic $B_* \geq \max\{1, 4K11^L\}$, then set $B_{i+1} = V(B_i)$. Set $d_0 = B_{m-1}$. All chosen scales are dyadic, and each divides d_0, hence also every multiple d of d_0. The coverage fraction is at most $1/2$. At an uncovered address, flipping the required source bit changes the desired output but none of the table's arguments. This contradicts correctness on the full input cube. `Compiler.no_bounded_path_transpose_of_dvd` gives this statement with an explicit d_0; `Compiler.no_bounded_path_transpose` derives the dyadic case. □

The formal circuit model also permits nonlinear gates between stages. A depth-L, fan-in-q node applies a permitted stage to each child's output and then an arbitrary address-dependent Boolean table. Leaves apply public unary tables to the original input, allowing constants as well as copies. The exact recursive unrolling has q^L paths, each of length L. Consequently the same exclusion holds for this circuit model, at every positive multiple of the corresponding threshold for $K = q^L$. This follows from `Circuit.unroll` and `Circuit.no_bounded_circuit_transpose_of_dvd`. The unrolling is proved from the circuit evaluation rule, rather than supplied as a dependency assumption.

The theorem does not allow the input to select the interval family or the paths. Nor does it cover arbitrary cross-address computation, increasing depth or fan-in, or an unrestricted tape implementation. It therefore excludes a compiler class, not the unrestricted converse WORD ⇒ BIT.

5 Adding synchronous nonlinear scans

The previous argument concerns permutations and pointwise Boolean gates. We now allow information to travel along a bank through a carried state. The resulting obstruction uses one global record of exceptional bits and scan boundary states. It does not require the intermediate banks to remain independent or linearly encoded.

Definition 5.1 (Public scan program)**.** A program has q banks of P bits and L layers. Each initial bank is either a copy of the raw input or a fixed public bit string. Each layer first applies one permitted stage from Definition 4.1 to each bank, then scans all banks synchronously from left to right or from right to left. At an address the scan reads the q old bits and its current state, writes q

new bits, and updates the state. The transition rule may be any public address-dependent nonlinear function.

The state is carried between layers. Its finite alphabet has at most 2^s values, for an integer $s \geq 0$, and its initial value is fixed. The stages, directions, transition tables and initial constants are independent of the input. The output is one designated final bank.

`RawScanProgram` gives these raw-address semantics. Grouping a scan into analytical bins preserves every output bit and its terminal state by `RawScanProgram.run_onGrid`. In particular, the program is fixed before the proof selects a bin scale. Public choices may vary with d; the theorem below fixes only q, L, s.

We first isolate a counting argument. The input ranges over the full Boolean cube, with its coordinates partitioned into public source bins. A *named demand* is an original input bit x_v, together with a public set A_v of source bins. A helper $H(x)$ determines that demand if equal helper values and equal input bits in A_v force equal values of x_v, for every pair of inputs.

Lemma 5.2 (One shared helper)**.** *Let F be a set of distinct original input coordinates, each with a named demand determined by one common helper and its own set A_v. Suppose each demand's source bin is absent from A_v and $|A_v| \leq D$ for $v \in F$. If the helper has at most 2^h possible values, where $h \in \mathbb{N}$, then*

$$|F| \leq (D+1)h. \tag{2}$$

Only demands in F need to be determined.

Proof. Order the source bins uniformly at random. Select $v \in F$ when its source bin precedes every bin in A_v. Its selection probability is $1/(|A_v|+1) \geq 1/(D+1)$.

Fix an order and restrict to inputs whose nonzero coordinates are all selected demands. Two such inputs with equal helpers agree on every selected bit: work backwards through the bin order. A selected demand's ancestor bins come later, so their selected bits have already been recovered; their unselected bits are zero. Local determination now recovers that demand. No selected demand uses its own bin as local side information. Thus the helper is injective on this subcube. If it has t selected coordinates, then $2^t \leq 2^h$, giving $t \leq h$. Averaging yields $|F|/(D+1) \leq h$. □

This is the acyclic-side-information method from index coding [2, Theorem 7], specialized to public bins. We do not claim a new information principle. The Lean result `helper_named_demands_bound` uses one range-cardinality budget, without assuming linearity or independence between helper fields. It formalizes the averaging by an exact finite count of bin orders, followed by the descending-rank injection above. The budget h is an integer; a real entropy budget is not asserted. The scan-specific work is to construct this common helper and prove the required public ancestry bounds.

For a scale $B \mid d$, put $n = P/B$ and record the current bit at every bad source position before each layer's permutations, in every bank. Let their total number be E_B. Also record the actual state entering each of the n bins in every scan, in its scan direction. This includes the inherited initial state. A scan's terminal state is not needed to replay its outputs; if another layer follows, that state is recorded as the next scan's initial state. The complete record has at most

$$2^{E_B + sLn} \tag{3}$$

possible values. This is a cardinality upper bound; the recorded fields can be dependent.

Given this record, a final output bin depends on at most $D = (11q)^L$ original bins. The induction uses at most 11 good predecessor bins per permuted bank and the scan's entering state. A scan's output on one bin is determined by that state and the old tuples in the bin, in either direction.

Both local replay and the entire layered reconstruction are proved from the transition recurrence in `Scan.output_congr_of_record` and `ScanProgram.run_congr_on_ancestry`. The ancestor sets are public and independent of the record.

Theorem 5.3 (Finite scan bound)**.** *Suppose a public scan program transposes every $d \times d$ bit matrix, where $d \geq 1$. For any $m \geq 1$ positive divisor scales $B_i \mid d$ satisfying $V(B_i) \leq B_j$ for $i < j$, there is a common scale $B = B_i$ such that*

$$\frac{P - DP/B}{D+1} \leq \frac{qLP}{m} + \frac{sLP}{B}, \qquad P = d^2, \quad D = (11q)^L. \tag{4}$$

Proof outline. Each output bin requests B bits from distinct source bins under transpose. At most D of them come from its own ancestor bins, leaving at least $P - DP/B$ far demands overall. Apply Lemma 5.2 with the complete record in (3). This gives the one-scale inequality with E_B in place of qLP/m.

The one-stage volume bound, summed over all q banks and L layers, gives $\sum_i E_{B_i} \leq qLP$. Choose one scale attaining the average. No volume is pulled back through nonlinear scans. The compiled endpoint is `RawScanProgram.exists_scale_transpose_bound`. □

Corollary 5.4 (Bounded-state scan exclusion)**.** *For fixed q, L, s, there is a positive dyadic integer d_0 such that no public scan program with those resource bounds transposes every $d \times d$ bit matrix at any positive dimension divisible by d_0. In particular, all sufficiently large dyadic dimensions are excluded. The layouts and transition tables may vary with the dimension.*

Proof outline. Choose a positive number of scales with $m \geq 8(D+1)qL$. Choose a dyadic first scale satisfying

$$B_0 \geq \max\{2D,\, 8(D+1)sL,\, 2\}, \qquad B_{i+1} = V(B_i).$$

Take $d_0 = B_m$. Every selected scale divides d_0, hence also every positive multiple d of d_0. The left side of (4) is at least $P/[2(D+1)]$. Each term on the right is at most $P/[8(D+1)]$, so their sum is at most $P/[4(D+1)]$, a contradiction. `RawScanProgram.not_transposes_of_threshold_dvd` proves the positive-multiple exclusion, and `RawScanProgram.not_transposes_large_dyadic` derives the dyadic case. `RawScanProgram.not_transposes_of_scales` states the finite conditions directly. □

The carried state and its boundary records are charged explicitly. The theorem permits arbitrary nonlinear transition tables, but still requires fixed public geometry, a bounded number of banks and layers, and a bounded state alphabet. It does not describe a general multitape machine with independently moving heads or growing auxiliary workspace. A construction using those freedoms is not excluded by this result.

6 Consequences and remaining questions

The reduction makes a linear-time dyadic bit transposer sufficient for canonical word transposition. Its contrapositive says that failure of linear-time canonical word transposition implies failure of linear-time dyadic bit transposition. The reverse transfer would require $\mathsf{WORD} \Rightarrow \mathsf{BIT}$, which remains open. No general quantitative lower-bound transfer is asserted.

The multiscale argument isolates a common obstruction: a short composition of whole-entry operations and simple rearrangements has limited fine-scale spread unless it traverses a block whose scale is exceptional. Disjointness bounds the total volume of these exceptional blocks across sufficiently separated scales. To cover synchronous scans, path counting must be supplemented

by a bound on information carried between addresses. For the synchronous scan grammar, one shared record supplies the exceptional values and scan boundary states, after which acyclic decoding charges the missing named input bits. The finite bound quantifies this cost: a helper of h bits can support at most $(D+1)h$ demands whose own bins are absent from ancestor sets of size at most D. The count is derived even when the intermediate bank encodings are nonlinear.

The state restriction matters. The theorem permits arbitrary nonlinear transition tables, but fixes the number of banks, layers, and state bits while the dimension grows. It does not permit an unbounded workspace to travel through a scan without being charged. Nor does it cover arbitrary asynchronous head schedules, input-dependent block layouts, or expanding banks. Extending the barrier to any of these features requires an additional proof.

The construction program now has a proved sufficient target: a uniform linear-time dyadic bit transposer supplies the canonical word transposer through Theorem 3.1. Alternatively, a reverse compiler must use operations beyond the excluded grammars; allowing such operations is only a candidate route, not evidence of a construction. In either case, every movement, state update, temporary copy and reset must be charged in the native model. For the impossibility program, the next target is an accounting principle that continues to bound the shared information when local state or access schedules grow with the input. Neither a restricted negative result nor a short algebraic factorization alone settles the original transposition question.

A Formalization and reproduction

The Lean proofs are the mathematical proofs for the theorems in this paper. Human-readable proofs and outlines explain those proofs; they do not enlarge the checked statements. The declaration map in `verification/contracts.json` identifies each numbered result and its supporting native interfaces.

The new package shares the native machine model and supporting results of [4, 5] through a relative Lake dependency. It preserves the existing Lean and mathlib pins [6, 10, 11]. Shared definitions are imported, not copied and independently redefined. All native execution claims are proved against actual transition traces, including input preparation and final output.

The reproduction script builds the mapped modules, checks their aggregate imports, and audits every declaration owned by their local import closure. It records source hashes, elaborated endpoint types and the axiom dependencies. The permitted foundations are `propext`, `Classical.choice`, and `Quot.sound`. The checks reject additional axioms, admitted proofs and unchecked native decision procedures. These checks complement the review of the mathematical contracts and the English-to-Lean correspondence; compilation alone does not establish that correspondence.

The paper's README gives the build and verification commands. Clone the whole repository so the sibling Lean packages retain their relative paths. Generated local logs are ignored; the sources and reproduction tools contain no machine-specific paths. The manuscript build is separate from the Lean audit.

References

[1] Micah Adler, Nicholas J. A. Harvey, Kamal Jain, Robert Kleinberg, and April Rasala Lehman. On the capacity of information networks. In *Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 241–250. Society for Industrial and Applied Mathematics, 2006. URL: https://www.math.uwaterloo.ca/~harvey/Publications/Capacity/Capacity-Soda.pdf.

[2] Ziv Bar-Yossef, Yitzhak Birk, T. S. Jayram, and Tomer Kol. Index coding with side information. *IEEE Transactions on Information Theory*, 57(3):1479–1494, 2011. doi:10.1109/TIT.2010.2103753.

[3] Charles H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973. doi:10.1147/rd.176.0525.

[4] Hans Brende and Claude Opus 5.5 (Anthropic). Matrix transposition and the tape coding conjecture. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 5f99ef1a503f5d56585a3862b2778193b6d358f4. URL: https://github.com/HansBrende/papers/tree/5f99ef1a503f5d56585a3862b2778193b6d358f4/Matrix-Transposition-and-the-Tape-Coding-Conjecture.

[5] Hans Brende and GPT-6 Astra (OpenAI). Local coding barriers and the limits of tape geometry for matrix transposition. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 5faf6890e8d54e9d8377174f16c8a3762c42cbd4. URL: https://github.com/HansBrende/papers/tree/5faf6890e8d54e9d8377174f16c8a3762c42cbd4/Local-Coding-Barriers-and-the-Limits-of-Tape-Geometry-for-Matrix-Transposition.

[6] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction—CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi:10.1007/978-3-030-79876-5_37.

[7] David Harvey and Joris van der Hoeven. Integer multiplication is at least as hard as matrix transposition, 2025. Version 1, 28 March 2025. URL: https://arxiv.org/abs/2503.22848v1, arXiv:2503.22848.

[8] Walter W. Kirchherr. Transposition of an $l \times l$ matrix requires $\Omega(\log l)$ reversals on conservative Turing machines. *Information Processing Letters*, 28(2):55–59, 1988. doi:10.1016/0020-0190(88)90163-9.

[9] Sriram Krishnamoorthy, Gerald Baumgartner, Daniel Cociorva, Chi-Chung Lam, and P. Sadayappan. On efficient out-of-core matrix transposition. Technical Report OSU-CISRC-9/03-TR52, Department of Computer and Information Science, The Ohio State University, September 2003. Section 4. URL: https://www.csc.lsu.edu/~gb/TCE/Publications/SeqTranspose-TR0352.pdf.

[10] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. ACM, 2020. arXiv:1910.09336, doi:10.1145/3372885.3373824.

[11] The mathlib Community. Mathlib4. Formal mathematics library, Lean 4 version, 2026. Pinned revision d13f23b723b8a846827a245b89c10fc7d3f11612, release v4.34.1. URL: https://github.com/leanprover-community/mathlib4/tree/d13f23b723b8a846827a245b89c10fc7d3f11612.